



Week 15

✓ Virtual Memory Mangement

- Idea of Demand Paging
- valid-invalid bit
- page table when some pages are not in main memory
- Page Fault
- How os handle page fault
- Graph of Pgae Faults vs the number of frames

✓ Algorithm to replace frame when the memory is full

- FIFO Page Replacement
- Optimal Algorithm
- LRU Page Replacement

✓ Global vs Local Allocation

(in addition to algo, sometimes, we need to think about global and local allocation)

✓ Thrashing

(another thing you should know about the page replacement)

✓ Program Structure

- The working set

| Content

✓ Virtual Memory Mangement

Overview

(last week, we learn the way OS manage the real memory, and if OS work that way that means we have no chance to run the program that exceed the memory size tht we currently have in the moment)



Idea of virtual memory (don't have to upload the entire program to main memory)

- that why modern os used the conceit of virtual memory, so the overview of virtual memory is that OS used the storage to simulate it to be part of the moery
- idea is taht part of your program will be loaded to memory, adn part of your program that annouce store in the storage
- for example, if your program divided into 10 pages, then maybe 5 pages are loaded into RAM, another 5 will be store in storage
 - and if your program work well with that 5 pages, no need to upload more pages
 - but if your program require the next page, OS will upload that page to the real memory. — because based on computer architecture that we used → actually the program and the data must be loaded to memory first (like the thing cpu need to use must be loaded to memory first)

Benefit

- Your program can be loaded and run faster
- You can run program even the memory size you have at the moment is less than your program size, because you don't need to upload the entire program.

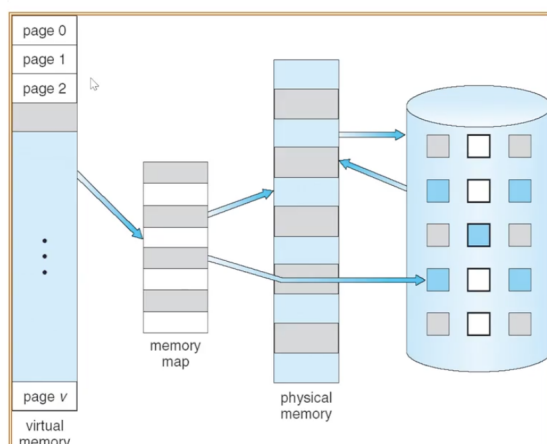
Background

- **Virtual memory** – separation of user logical memory from physical memory.
 - Only part of the program needs to be in memory for execution
 - Logical address space can therefore be much larger than physical address space
 - Allows address spaces to be shared by several processes
 - Allows for more efficient process creation
 - Virtual memory can be implemented via:
 - Demand paging
 - Demand segmentation
- In summary, logical address space that is the program size can be larger than physical address space
 - **actually there 2 major technique for virtual memory**
 - Demand paging (focus on this) (many os used this)
 - Demand segmentation
 - As we have learned, nowadays, we used segmentation and paging together, so even if tho the program divide into segmentations, inside segment → divide into pages
 - we can said that bts paging is used for manage your program

Segmentation fault → we can't used program that is out of your segmentation.

Idea of demand paging

Virtual Memory That is Larger Than Physical Memory



Overview Idea and how demand paging work

- Your program have, let said, page 0 to page v.
- And you can see that your RAM (physical memory) is smaleer than your program → so you can't load the entire program to memory
- (you can load part of the program into page table (memory map)) → and then if your program request a page that still not in page table, OS will upload the page to the memory, and update page table

Demand Paging

- Bring a page into memory only when it is needed
 - Less I/O needed
 - Less memory needed
 - Faster response
 - More users
 - Page is needed ⇒ reference to it
 - invalid reference ⇒ abort
 - not-in-memory ⇒ bring to memory
 - **Lazy swapper** – never swaps a page into memory unless page will be needed
 - Swapper that deals with pages is a **pager**
-
- We need to understand this one → when program need the page
 - if the page already in memory → the program can used it without problem
 - if the page not in mamory, there are 2 cases when you refer to a page
 - invalid reference(error) — we not focus on this case
 - not-in-memory — when the program want the page, and it can't find the page → it mean the page not in memory

The idea of Demand Paging

- **Lazy swapper**
 - The os will never think in advacned — i should load this page first —
 - It only load the page when it needed

- Part of OS that bring in bring out the page from storage to emory → swapper or pager

Valid-Invalid Bit

Valid-Invalid Bit

- With each page table entry a valid-invalid bit is associated (**v** ⇒ in-memory, **i** ⇒ not-in-memory)
- Initially valid-invalid bit is set to **i** on all entries
- Example of a page table snapshot:

Frame #	valid-invalid bit
	v
	v
	v
	v
	i
...	
	i
	i

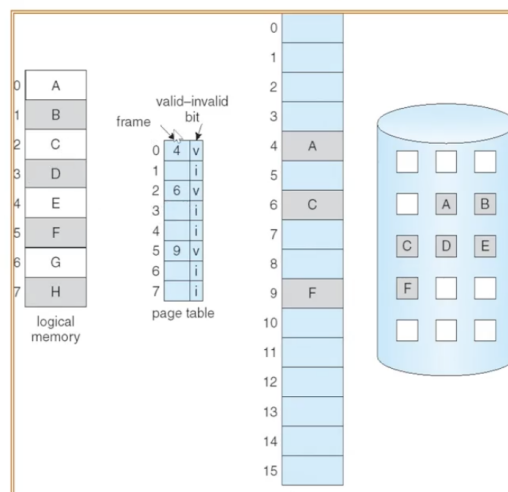
page table

- During address translation, if valid-invalid bit in page table entry

- Here, we still use page table to map the page
- we still use valid and invalid bit, the same as we did in real memory
- Major different (virtual, real memory)
 - for invalid, in the real memory mean error
 - for virtual, invalid can both mean error, or that page is not in the memory at that time. (just load it) (in this class, focus on not in memory)
- Idea is → When a program refer to the page that not in memory , it is called page fault.

Page table when some pages are not in main memory

Page Table When Some Pages Are Not in Main Memory



- page 1 → invalid because it haven't been loaded

📌 Page Fault

- So, what happen when page fault occur(program refer to the page tha tis not in memory)

Page Fault

- If there is a reference to a page, first reference to that page will trap to operating system:

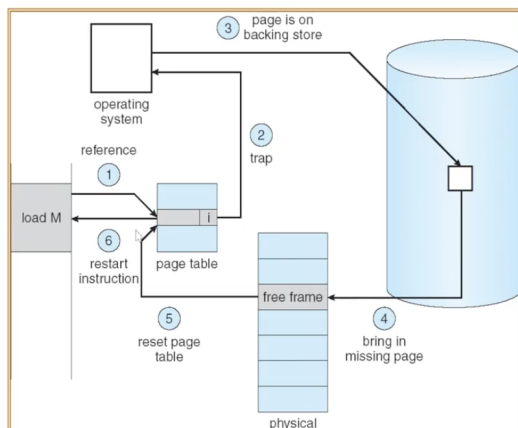
page fault

- Operating system looks at another table to decide:
 - Invalid reference ⇒ abort
 - Just not in memory
- Get empty frame
- Swap page into frame
- Reset tables
- Set validation bit = **v**
- Restart the instruction that caused the page fault

📌 How OS handle page fault

Got program page, and update page table

Steps in Handling a Page Fault



- Assuming that the program need to load data from this memory(M)
 - program want data in this address
- Os said I want to access this address → it go to page table
 - It find out that that page still not in memory
- When program refer to a page that is not in memory → interrupt occur → and when interrupt occur → OS work
- OS then find the page on the storage, and after that it load the pages to free frame in memory and update page table
- And then ask the program to run,execute the same command again
- this time no page fault occur 🎉

The thing that we need to consider is this case

- when a page is needed, a page is loaded into a free frame — the question is what if there is no free frame(memory is now full)
 - OS won't tell use rthat it won't run
 - wen this happen — OS need to replace something.
- Os has algorithm to replace the frame — the idea is we can analyze algorithm in the sense that A is better than B if the page fault rate of A is better than B (OS need to decide to replace some frame)
- Many algo OS used to replace the frame here
 - the lower the page fault rate → the better the algorithm

What happens if there is no free frame?

- Page replacement – find some page in memory, but not really in use, swap it out
 - algorithm
 - performance – want an algorithm which will result in minimum number of page faults
- Same page may be brought into memory several times

Page Replacement Algorithms

- Want lowest page-fault rate
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string

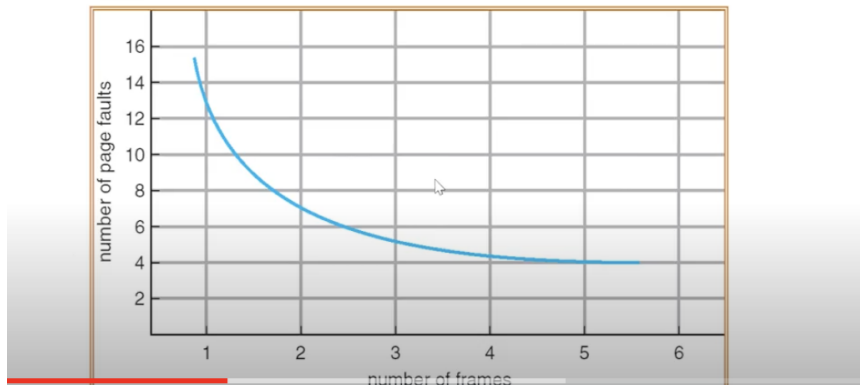
When a new algorithm is developed → the way to test is we need to simulate the page that the program need to use → and that order of a page is call "page reference string" — fake data

- in real practice we never knpw this page reference string (we don't know in advanced which page program will called)
- this ref string - used for testing algo only



Graph of Page Faults vs the number of frames

Graph of Page Faults Versus The Number of Frames

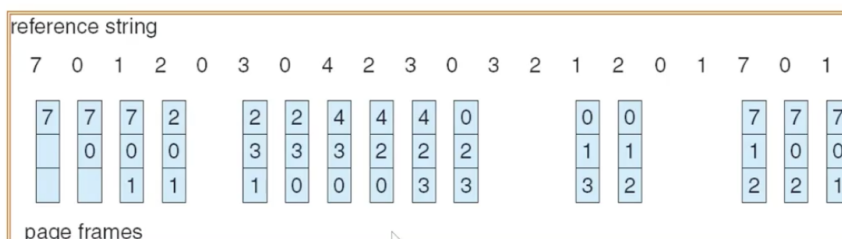


- number of frames → increase memory size
- increase memory size can reduce page fault rate (int eh begining)
 - after some point, there no longer help
- when you face performance of your machine → you think first → upgrade the RAM → but at some point, it not help — should think of other way
 - maybe because bottleneck is about storage (SSD, network) maybe update hardware speed to get the network faster

✓ Algorithm to replace frame when the memory is full

📌 FIFO Page Replacement

FIFO Page Replacement



- this is very simple, some OS might use it, but it not the best one

So, the point is, prof will ask, for this fake ref string → when it run it will start from page 7, 0, 1n

- using this page ref string, and assume there are only 3 frames in memory → how many page fault will occur when this program run from start to finish
- we assume that before this program run → 3 frames are available
- when the first page is needed, because the frame is empty → the first page fault occur → 7 is put in
- 0 → not in → page fault occur → put 0 in
- 1 → not in → page fault occur → put 1 in

for this first 3 pages, 3 page fault occur

- 2 is needed, now the memory is full → 2 must replace some pages



FIFO → which one come first be replaced first (in this case 7 come first)

- 2 replaced 7
- 0 is already in the frame → no page fault → no replacement
- 3 is needed → 3 replace 0

so on and so forth....

OS keep doing this, when we finish → count number of page fault → 15

- if we used FIFO, 15 page fault occur when program start running until it finish
- count the rectangle box



Optimal Algorithm

- if we can implement this, it will give best page fault rate, but the point is we can never implement it.

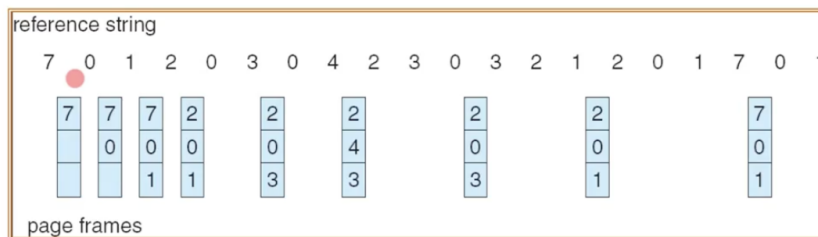
Optimal Algorithm

- Replace page that will not be used for longest period of time
- How do you know this?
- Used for measuring how well your algorithm performs



- Why do we have to study the algo that can never be implement
 - use as ref → when develop new → very close → good

Optimal Page Replacement



- the first 3 pages → same as FIFO
- 2 replace waht



the idea of optimal algo → look at the rest of the page string, find out which page will be call the last, (based on 7 0 1 → 7 will be called the last)

- 2 replaced 7
- 0 → no page fault
- 3 (2, 0, 1) → replace 1
- OS keep doing this → 9 page fault occur

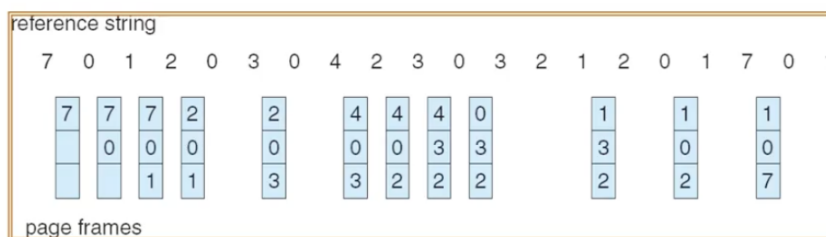
Algo can't be implemented → bacause in real, we never know ref string — now is interactive, not batch

- but the point is this one can't be implemented, which one is better than FIFO for OS to use and not have statistic

📌 LRU Page Replacement (less recently used)

(opposite way of optimal page replacement) (take a look at the past) (keep record of the thing used) (timestamp)

LRU Page Replacement



- the first 3 page → the same (3 page fault)
- 2 need to replace some pages

👉 (LRU) → take a look for the page in the frame, which one is not access for the longest time (not used for the longest time)

- in this case 7 is not used the longest time → 2 replaced 7
- 0 → no page fault
- 3 → (2, 0, 1) which one → replace 1
- 0 → no page fault
- 4 → (2, 0, 3) which one → replace 2

So on and so forth.....

- page fault = 12, many OS used LRU, page replacement algo

✅ Global vs Local Allocation

Global vs. Local Allocation

- **Global replacement** – process selects a replacement frame from the set of all frames; one process can take a frame from another
 - **Local replacement** – each process selects from only its own set of allocated frames
-
- Global replacement(allocation) — when a frame need to be replace, os can choose to replace anyframe
 - for example, if frame 0 1 2 is assign for process A, 3 4 5 is for process B
 - process B now need some other pages, so, the memory is full, for global replacement, OS may choose to bring the page and replace the frame that belong to process A.
 - OS can replace any frame in memory, OS doesn't care which frame is for which process. OS can choose any frame to replace
 - Local replacement - OS will replace only the frame that are assigned to for specific process only, it will not replace the frame from some other process.
 - **Which one is better?**
 - OS always used paging, when memory is full, os need to replace some frame in the memory.
 - The advantage of global is that, it seem like process has more memory to used → because OS can choose any frame.
 - problem of global → page fault rate of process sometimes never occur because of the process it self, but because of some other process.
 - for ex: process A used 0 1 2, process B used 3 4 5. When process B need something, OS replace frame of page A. and if process A need page that just be replaced → A get page fault that cause by page B. Process itself can't control the page fault thing of themselves.
 - page fault rate cannot be calculated by the process itself, because the frame can be replaced by some other process.

- But for Local replacement, we can calculate page fault rate for process itself, but we have limited memory, we have to use the memory that is assigned to process only. (the page, the frame something)
- this 2 condition can be included in algorithm to make the variation

✓ Thrashing

Thrashing

- **Thrashing** ≡ a process is busy swapping pages in and out
- If a process does not have "enough" pages, the page-fault rate is very high. This leads to:
 - low CPU utilization
 - operating system thinks that it needs to increase the degree of multiprogramming
 - another process added to the system

What is Thrashing?

- the situation that memory is full, everytime a new page is needed, it always causes page fault. because memory doesn't have enough frames to store page
- the point is when thrashing occurs → it means OS needs to busy swapping pages in and pages out from storage and memory. when OS does that, it means CPU may now be idle. Because OS can't run any process.

So, if OS can't detect that thrashing occurs → OS may think CPU is now idle, it should load more programs to run → and when it loads more programs → requires more memory.

- OS may need to do the way to find the way of thrashing

Thrashing (Cont.)

- ❑ Operation becomes inefficient
- ❑ Caused when a page is removed from memory but is called back shortly thereafter
- ❑ Can occur across jobs, when a large number of jobs are vying for a relatively few number of free pages
- ❑ Can happen within a job (e.g., in loops that cross page boundaries)

How to detect thrashing



The working set

The Working Set

- ❑ **Working set:** Set of pages residing in memory that can be accessed directly without incurring a page fault
 - ❑ Improves performance of demand page schemes
 - ❑ Requires the concept of "locality of reference"
- ❑ **System must decide**
 - ❑ How many pages compose the working set
 - ❑ The maximum number of pages the operating system will allow for a working set

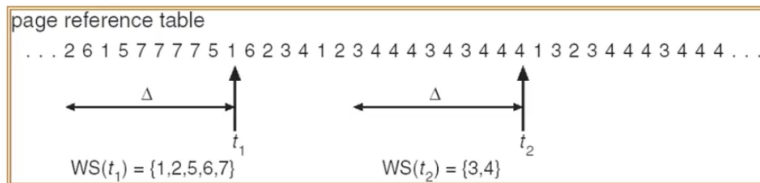
Working-Set Model

- ❑ $\Delta \equiv$ working-set window $\equiv$ a fixed number of page references
Example: 10,000 instruction
- ❑ WSS_i (working set of Process P_i) =
total number of pages referenced in the most recent Δ (varies in time)
 - ❑ if Δ too small will not encompass entire locality
 - ❑ if Δ too large will encompass several localities
 - ❑ if $\Delta = \infty \Rightarrow$ will encompass entire program
- ❑ $D = \sum WSS_i \equiv$ total demand frames
- ❑ if $D > m \Rightarrow$ Thrashing
- ❑ Policy if $D > m$, then suspend one of the processes

- working set here is a set of window that cover the pages that each process used at a time, and then os has to calculate the frame or memory that it has

at that time.

Working-set model



Assuming that, this is page ref for 1 process, and if window size is 10

- at t_1 , assuming that process 1 need 2 - 1 pages (10 pages)
 - os know that process p1 require only 5 frames.
- at time t_2 , another 10 pages from 3 - 4
 - proces p1 need only 3 and 4
- delta will change by the time os calculate
- So, how can we detect
 - Assuming that at the same time , time t_1 , process p2 said that i want my page number 1 2 3 8 9 0 2 4 3 2 → $WS(t_1) = 0, 1, 2, 3, 4, 8, 9$ (list of process it needed at time t_1) (we need 7 pages) → this mean at time t_1 , process p2 require 7 frames, process o1 require 5 frames
 - this mean if we have 12 frames at least in memory → thrashing will never occured at time t_1 .
 - p1 - 5 frames, p2 need 7 frames → we need 12 → if we have 12 free frame in memory, thrashing won't occur in time t_1
 - but if we have less than 12 free faems → thrashing can occured.
- So, for each time working set of process changes, OS has to calculate to check for thrashing.

Keeping Track of the Working Set

- Approximate with interval timer + a reference bit
- Example: $\Delta = 10,000$
 - Timer interrupts after every 5000 time units
 - Keep in memory 2 bits for each page
 - Whenever a timer interrupts copy and sets the values of all reference bits to 0
 - If one of the bits in memory = 1 $\Rightarrow$ page in working set
- Why is this not completely accurate?
- Improvement = 10 bits and interrupt every 1000 time units

✓ Program Structure

Program Structure

- Program structure
 - `int[128,128] data;`
 - Each row is stored in one page
 - Program 1

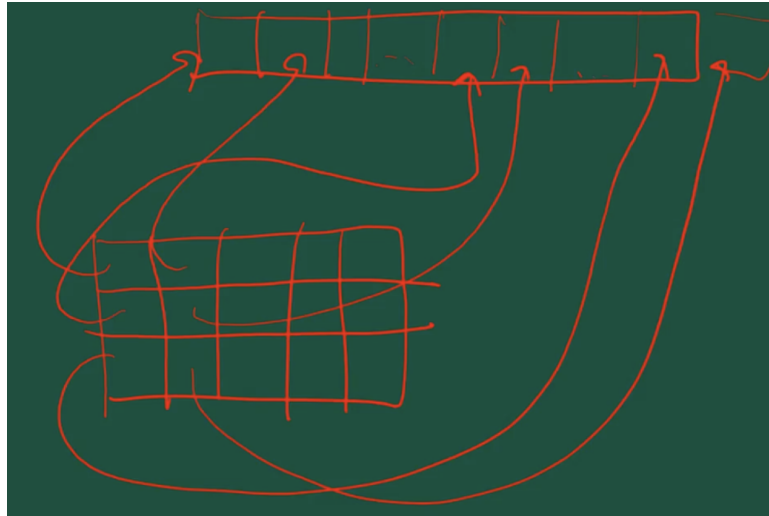
```
for (j = 0; j < 128; j++)
  for (i = 0; i < 128; i++)
    data[i,j] = 0;
```

128 x 128 = 16,384 page faults
 - Program 2

```
for (i = 0; i < 128; i++)
  for (j = 0; j < 128; j++)
    data[i,j] = 0;
```

128 page faults

- if we need to initialize data that is 2 dim array, which one that we should use
- if it java $\rightarrow$ second one is better, we can tell by taking look of number page fault.
- Assume, each row require 1 page.
- if we use row major array and use program 1

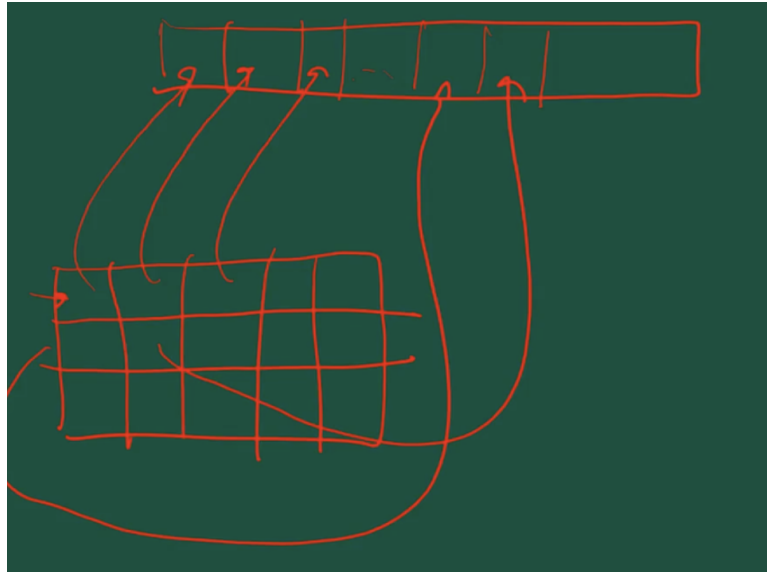


- everytime, you access element, page fault occur everytime
- but if using second way → go one row first, go to next page

Have you ever heard row major array and column major array

There are 2 types of array in programming language.

- the point is → when we declared 2 dimensional array → actually we take a look at array just like table
- but the point is → OS doesn't manage memory this way → OS always see the memory in linear something like the pic
- so, for 2D array, we see this as table, but os manage memory just like linear
- so, this mean, this slot must match to slot of computer?
- For java, we used row-major array → which mean column change first, and then row
 - so if the programming use row major, the mapping is just like
 - map row first



change by col first, finish row, change to next row

What about col major

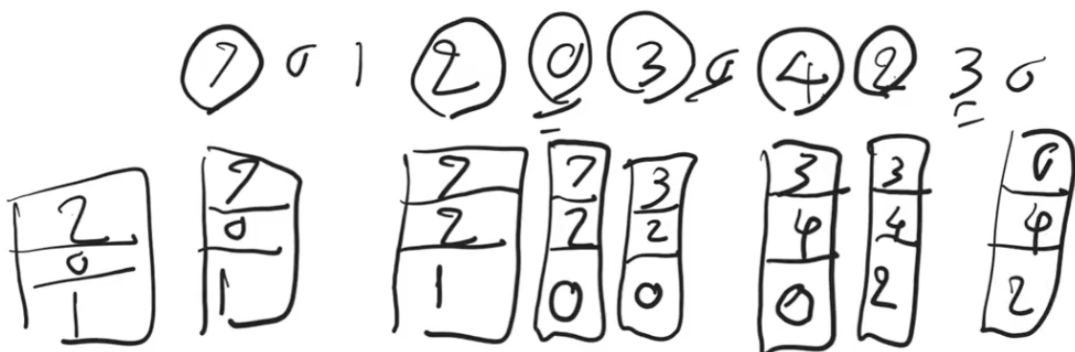


Quiz

1. For virtual memory concept, logical address space can be larger than physical address space.
 - True
2. For virtual memory concept, the entire program does not need to be loaded at the same time
 - True
3. For demand paging virtual memory concept, the page table is still used for logical address to physical address translation.

- True
- A bit in the page that is used to determine that a page is in a frame or not is called
 - valid-invalid bit
 - Reference to a page that is not in the memory will cause
 - page fault
 - If the thing in the previous question occurs, an OS will do the following excepts ____
 - change the value of valid-invalid bit to invalid.
 - (it will get empty frame, swap page into frame, update page table, restart the instruction that caused the page fault)
 - If a page fault occurs and there is no free frame available, OS will terminate the program
 - False
 - In general, if we add more memory to the system the page fault rate will decrease
 - True
 - Question

9. If the memory has 3 frames, and now page 2, 0, and 1 are loaded into these frames respectively. Assume the page reference string of the remaining program is as follows: 7 0 1 2 0 3 0 4 2 3 0. Also, assume that the FIFO page replacement is used. If page 7 is referenced, the page that will be replaced is _____.



- 2

10. Question

10. If the memory has 3 frames, and now page 2, 0, and 1 are loaded into these frames respectively. Assume the page reference string of the remaining program is as follow 7 0 1 2 0 3 0 4 2 3 0. Also, assume that the FIFO page replacement is used. How many page faults will occur when the process starts executing the remaining parts of the program.

- 7

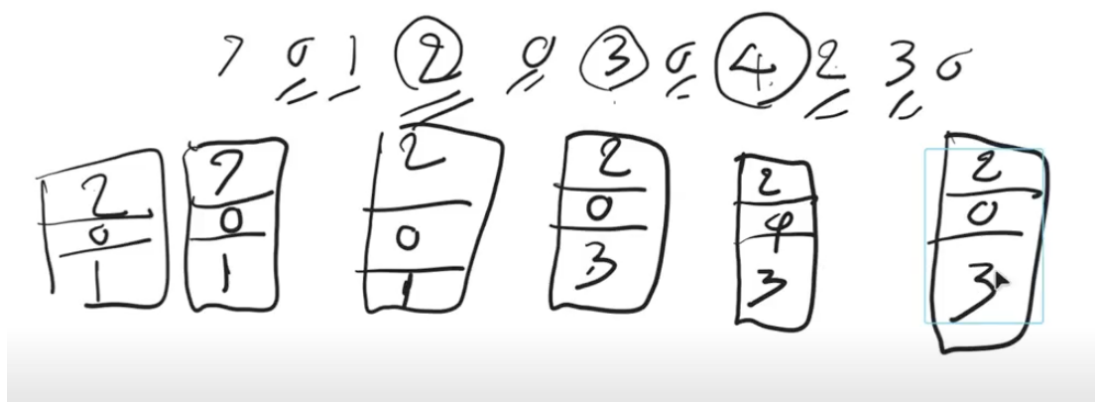
11. Question

11. If the memory has 3 frames, and now page 2, 0, and 1 are loaded into these frames respectively. Assume the page reference string of the remaining program is as follow 7 0 1 2 0 3 0 4 2 3 0. Also, assume that the Optimal page replacement is used. If page 7 is referenced, the page that will be replaced is _____.

- 7 will replaced 2

12. How many page fault will occur

- 5



- dont count first block

13. Optimal apge replacement is the algorithm that most OSes use since it gives teh minimum page fault rate

- False

14. Question

14. Assume that the memory has 3 frames and page reference string is as follows; 7 0 1 2 0 3 0 4 2 3 0. The page replacement algorithm is LRU. How many page faults will occur when this program is executed?



- 9

15. The frames that will be selected as replacement frames for a process will be chosen from only the frames that OS allocated to the process.

- False (we have global and local)

16. If a system is busy swapping pages in and out, it can be said that ____ occurred.

- thrashing

17. If an OS cannot detect the situation in the previous question, it may load more processes into the system since the CPU is not well utilized.

- True

18. Question

18. Assume that working set window is 10. At the time t1, assume that the process p1 has the following page reference string 2 5 1 5 7 7 7 7 5 1. Assume that the process p2 has the following page reference string 1 2 0 3 4 5 6 6 6 7. How many minimum numbers of free frames must be available to avoid thrashing?

- we have 10 at t1, p1 → the working set is 4 (2, 5, 1, 7)
- p2 → 8 (1, 2, 0, 3, 4, 5, 6, 7)
- $8 + 4 = 12$
- ans → 12 (no intersect)

19. Question

19. From the following Java program public class Main { public static void main(String[] args) { int [][] a = new int[10][256]; for(int j = 0; j < 256; j++) { for(int i = 0; i < 10; i++) { a[i][j] = 0; } } } Assume that one row of an array requires the space of 1 page. How many page faults will occur for the array initialization as shown in the code?

- 2560

20. Question

20. From the following Java program; public class Main { public static void main(String[] args) { int [][] a = new int[10][256]; for(int i = 0; i < 10; i++) { for(int j = 0; j < 256; j++) { a[i][j] = 0; } } } Assume that one row of an array requires the space of 1 page. How many page faults will occur for the array initialization as shown in the code?

- 10 (cause opage fault, when you change the row)

Exam

- after midterm
- understand what does each line do → fill in program
- code can be both java and C.
- semaphore c, java?